

Facultad de Ingeniería de Sistemas e Informática

Informe sobre Análisis del Problema

Alumnos:

Parque Niquen Angel Luis
Chanco Chambergó Jhymm Amir
Cuba Perez Jhosep Jhordan
Tacza Paredes Diego Shandel

Docente:

Osorio Contreras Rosario Delia

Huancayo - 2026

Capítulo 1: Análisis del Problema

1. Descripción del problema

En los entornos informáticos, la ejecución simultánea de diversas aplicaciones requiere que el sistema operativo administre correctamente los recursos del hardware. Cuando las tareas se ejecutan sin un control adecuado, pueden aparecer problemas importantes: la Unidad Central de Procesamiento (CPU) puede saturarse con procesos menos relevantes mientras las tareas prioritarias quedan en espera, generando inanición; además, la memoria RAM puede asignarse de manera desorganizada, ocasionando fallos por falta de espacio o desbordamientos.

¿Qué se busca resolver con el sistema?

Este proyecto tiene como objetivo simular el funcionamiento interno de un sistema operativo mediante un software centralizado capaz de solucionar tres necesidades principales:

- ***Organización:*** Registrar, buscar y ordenar dinámicamente las aplicaciones activas para mantener un control adecuado de cada proceso.
- ***Priorización:*** Asegurar que los procesos críticos del sistema sean atendidos antes que las tareas secundarias del usuario, evitando retrasos en el uso de la CPU.
- ***Control de memoria:*** Supervisar de manera precisa el consumo de memoria RAM de cada proceso, permitiendo asignar y liberar recursos de forma ordenada y eficiente.

-

2. Requerimientos del sistema

Funcionales

- **RF-01 (Registrar):** El sistema permite al usuario ingresar nuevos procesos de manera dinámica con un ID único, nombre descriptivo y nivel de profundidad inicial.
- **RF-02 (Eliminar):** El sistema debe permitir que el usuario elimine un proceso de los registros hechos cuando este termine o sea cancelado.
- **RF-03 (Buscar):** El sistema debe permitir hacer consultas al usuario y localizar los datos de uno de los procesos utilizando su ID o el nombre como criterio de búsqueda.
- **RF-04 (Modificar Prioridad):** El sistema debe permitir que el usuario actualice el nivel de importancia de cualquiera de los procesos registrados en el sistema para reordenar la ejecución de estos.
- **RF-05 (Planificar CPU):** El sistema debe enviar las tareas a la cola de espera, encargándose de que el proceso con mayor prioridad del registro sea apartado.

- **RF-06 (Gestionar Memoria):** El sistema se encargará de asignar bloques de memoria a las tareas en ejecución y desasignarlos de forma secuencial una vez que ya no se requieran.
- **RF-07 (Mostrar Estado):** El sistema debe desplegar una interfaz donde se refleje la condición actual de los procesos registrados, la situación de la cola de la CPU y el estado del mapa de memoria.
- **RF-08 (Persistencia):** El sistema debe permitir guardar los datos en un archivo local. Así mismo, también debe permitir cargarlos al iniciar la aplicación.

No funcionales

- **RNF-01 (Herramienta):** El código será desarrollado por completo en lenguaje C++ utilizando estrictamente el entorno **DEV C++** como compilador principal.
- **RNF-02 (Interfaz):** El usuario deberá tener una interfaz clara y organizada mediante un menú de opciones en la consola del sistema.
- **RNF-03 (Validación de Datos):** El sistema obligatoriamente debe validar cada entrada que realice el usuario evitando que el programa colapse.

3. Estructuras de datos propuestas

Para representar el flujo del sistema operativo en la memoria RAM, se define una estructura base (struct) que actuará como el contenedor de cada proceso.

```
struct NodoProceso {
    id;                // Identificador único del proceso
    string nombre;     // Nombre de la aplicación
    int prioridad;     // Nivel de importancia (ej: 1 = Alta, 3 = Baja)
    int memoria;       // Cantidad de bloques de memoria consumidos
    NodoProceso* siguiente; // Puntero de enlace al próximo nodo
};
```

A partir de esta struct base, se unirán las tres estructuras dinámicas lineales obligatorias:

- Lista Enlazada Simple: Para el registro general y manipulación de procesos.
- Cola de Prioridad: Para la simulación del despacho de tareas a la CPU.
- Pila Dinámica (Stack): Para la asignación jerárquica de bloques de memoria.

4. Justificación de la elección

Lista: Elegimos una lista enlazada porque en un sistema operativo los procesos se crean y cierran a cada rato. Un arreglo estático nos limitaría a una cantidad fija de tareas o nos haría desperdiciar memoria RAM si está vacío. En cambio, la lista crece y se achica sola en tiempo de ejecución; solo movemos punteros para meter o sacar nodos sin mover todo lo demás

Cola: Elegimos una cola de prioridad porque la CPU no puede atender a ciegas por orden de llegada, como en un FIFO clásico, ya que las tareas críticas del sistema operativo se congelarían por culpa de tareas comunes del usuario. Con esta cola, evaluamos la importancia del proceso al momento de insertarlo; así nos aseguramos de que al ejecutar y desencolar, la cabecera siempre

entregue primero la tarea más urgente.

Pila: Elegimos una pila para la memoria porque funciona bajo el principio LIFO: el último dato en entrar es el primero en salir. Cuando el sistema abre procesos o funciones, sus bloques de memoria se van apilando hacia arriba. Por eso, al cerrarlos, los últimos bloques asignados tienen que ser los primeros en liberarse; así mantenemos la memoria limpia y evitamos fugas de memoria.

También se utilizaron las siguientes funciones:

1. El Bucle while

Lo usamos porque no sabemos cuántos procesos hay guardados; nos permite recorrer los nodos y parar cuando el puntero llega a NULL. También lo usamos para validar: si ponen un ID negativo o repetido, el while bloquea el programa hasta que pongan un dato correcto.

2. El Condicional if / if-else

"Lo usamos para tomar decisiones según los punteros. Por ejemplo, con un if (Inicio == NULL) vemos si la lista está vacía para meter el proceso directo. Si no lo está (else), el programa ya sabe que debe buscar la posición correcta."

3. El Bucle do-while

"Lo usamos netamente para el menú de la consola. Sirve para que las opciones del sistema aparezcan en pantalla al menos una vez obligatoriamente, y se sigan repitiendo en bucle hasta que el usuario marque la opción de salir."

Capítulo 2: Diseño de la Solución

1. Descripción de estructuras de datos y operaciones:

El sistema se compone de tres estructuras dinámicas lineales interconectadas que operan sobre un nodo común (struct):

1. Lista Enlazada Simple (Gestor de Procesos): Organiza todos los procesos registrados de forma secuencial. Sus operaciones principales son insertar un nodo al final, buscar de forma iterativa (por ID o nombre) y eliminar un nodo reconfigurando los enlaces de los punteros vecinos.
2. Cola de Prioridad (Planificador de CPU): Administra el turno de ejecución. La operación de inserción examina la prioridad del proceso para colocarlo en la posición correcta (menor número de prioridad va más adelante). Al ejecutar, se realiza la operación de desencolar extrayendo siempre el nodo del frente.
3. Pila Dinámica (Gestor de Memoria): Controla el espacio RAM de forma vertical. Utiliza la operación push para apilar bloques de memoria asignados a un proceso activo y la operación pop para liberar el bloque ubicado exclusivamente en el tope de la pila.

Capítulo 3: Solución Final

1. Código limpio, bien comentado y estructurado.

2. Capturas de pantalla de las ventanas de ejecución con las diversas pruebas de validación de datos
3. Manual de usuario

Capítulo 4: Evidencias de Trabajo en Equipo

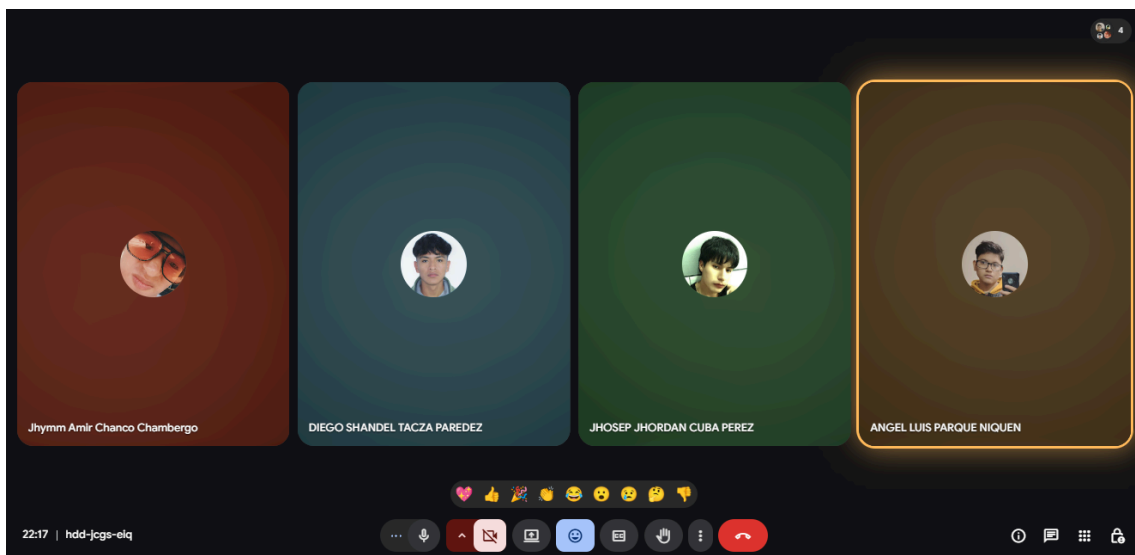
1. Repositorio con Control de Versiones (Capturas de Pantalla)

<https://github.com/AngelParque/Estructura-de-datos---trabajo-grupal.git>

2. Plan de Trabajo y Roles Asignados

Roles:

Integrantes	Roles
Chanco Chambergó Jhymm	Descripción del problema
Tacza Paredes Diego	Requerimientos del sistema
Parque Ñiquen Angel	Estructuras de datos propuestas
Cuba Perez Jhosep	Justificación



Reunión realizada el día 05-19-2026